

Итоговый проект

Python для разработки

ETL-пайплайн нормализации данных по доставкам

Витрины заказов и товаров (PySpark)

Стек: Docker Compose / PostgreSQL / Apache Airflow / PySpark

1. Исходные данные

Дана одна денормализованная таблица с 25 колонками и ~9.7 млн строк (34 Parquet-файла), содержащая информацию о доставках: заказы, пользователи, товары, курьеры, магазины — всё в одной плоской структуре.

Проблемы исходной структуры:

- Избыточность данных — данные пользователя, магазина, курьера дублируются в каждой строке
- Аномалии обновления — при изменении телефона нужно обновить все строки пользователя
- Аномалии удаления — удаление заказа приводит к потере данных о товаре
- Аномалии вставки — нельзя добавить товар без привязки к заказу

Атрибуты исходных данных:

Атрибут	Описание
order_id	ID заказа
user_id	ID пользователя
user_phone	Телефон пользователя
address_text	Адрес доставки
created_at	Дата создания заказа
paid_at	Дата оплаты
delivery_started_at	Дата начала доставки
delivered_at	Дата доставки
canceled_at	Дата отмены
payment_type	Тип оплаты
item_id	ID товара
item_title	Название товара
item_category	Категория товара
item_quantity	Количество единиц
item_price	Цена за единицу
item_canceled_quantity	Отмененное кол-во
item_replaced_id	ID товара-замены
order_discount	Скидка на заказ, %
item_discount	Скидка на товар, %
order_cancellation_reason	Причина отмены
driver_id	ID курьера
driver_phone	Телефон курьера
delivery_cost	Стоимость доставки
store_id	ID магазина
store_address	Адрес магазина

2. Выбор нормальной формы

Выбрана Третья нормальная форма (3НФ) как оптимальный баланс между:

- Устранением избыточности данных
- Целостностью данных (FK-связи)
- Удобством запросов (не слишком глубокая декомпозиция)

Правила перехода к 3НФ:

1НФ — первая нормальная форма

Все атрибуты атомарны, нет повторяющихся групп. Исходные данные уже в 1НФ.

2НФ — вторая нормальная форма

Устраняем частичные зависимости от составного ключа. Выделяем справочники: users, stores, drivers, items — их атрибуты зависят только от своего ID.

3НФ — третья нормальная форма

Устраняем транзитивные зависимости:

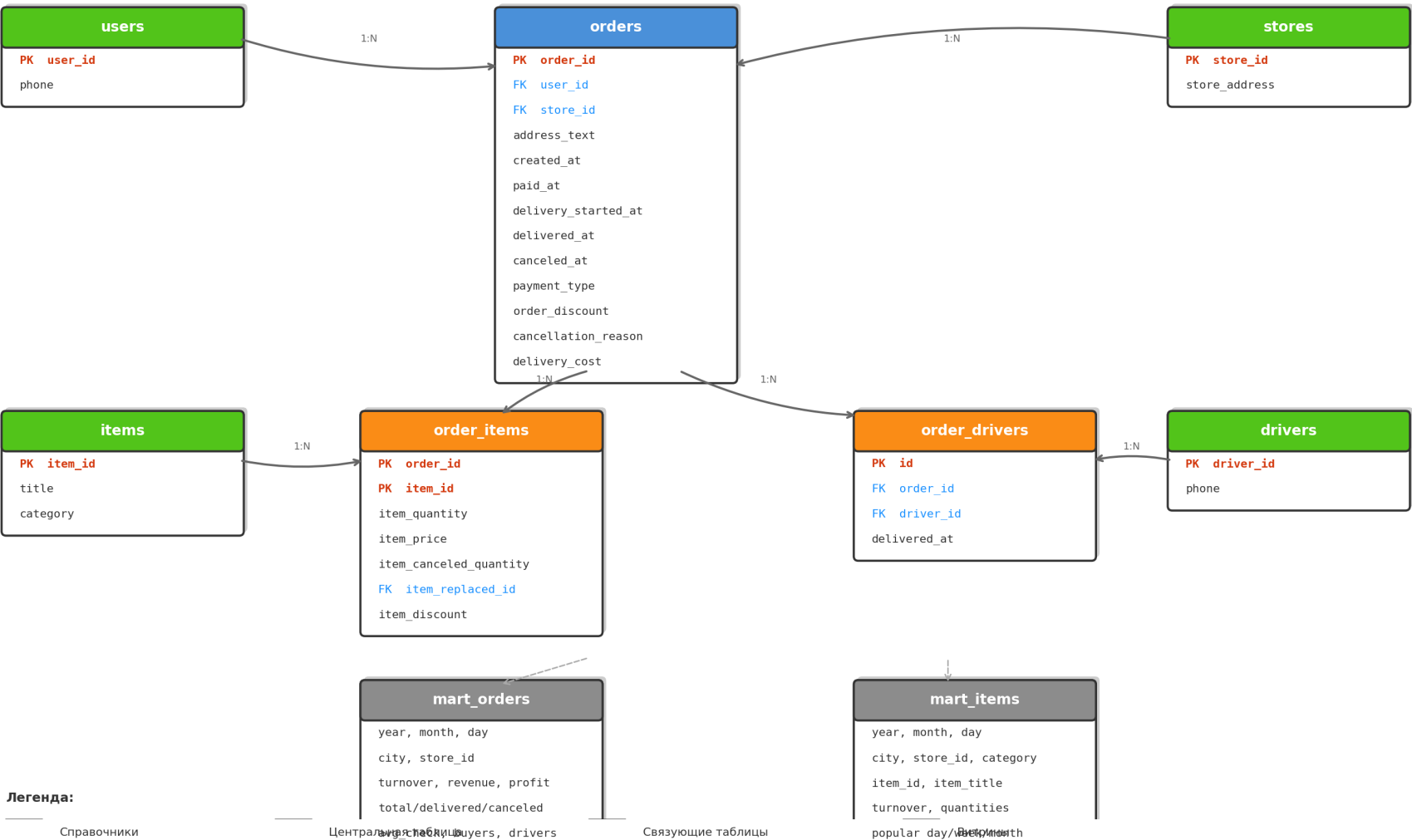
order_id -> user_id -> user_phone
order_id -> store_id -> store_address
driver_id -> driver_phone

Вынося эти атрибуты в отдельные таблицы, мы достигаем 3НФ.

3. Схема базы данных (ER-диаграмма)

Нормализованная схема БД (3НФ) — delivery

Схема: delivery | PostgreSQL 15



4. Декомпозиция на 7 таблиц

1. **delivery.users** — Справочник пользователей

Поля: user_id (PK), phone

Причина: user_phone зависит транзитивно: order_id -> user_id -> user_phone.

Записей: 31 910

2. **delivery.stores** — Справочник магазинов

Поля: store_id (PK), store_address

Причина: store_address зависит транзитивно: order_id -> store_id -> store_address.

Записей: 3

3. **delivery.drivers** — Справочник курьеров

Поля: driver_id (PK), phone

Причина: driver_phone зависит транзитивно: driver_id -> driver_phone.

Записей: 390

4. **delivery.items** — Справочник товаров

Поля: item_id (PK), title, category

Причина: item_title и item_category зависят транзитивно от item_id.

Записей: 18

5. **delivery.orders** — Заказы

Поля: order_id (PK), user_id (FK), store_id (FK), address_text, created_at, paid_at, delivery_started_at, delivered_at, canceled_at, payment_type, order_discount, order_cancellation_reason, delivery_cost

Центральная таблица с FK на users и stores.

delivered_at = время финальной доставки. Записей: 340 000

6. **delivery.order_items** — Позиции заказа

Поля: order_id (PK, FK), item_id (PK, FK), item_quantity, item_price, item_canceled_quantity, item_replaced_id (FK), item_discount

Связь many-to-many: заказы <-> товары. Составной PK (order_id, item_id).

При замене: обе записи (исходный + замена через item_replaced_id). Записей: 4 212 103

7. **delivery.order_drivers** — Назначения курьеров

Поля: id (PK, SERIAL), order_id (FK), driver_id (FK), delivered_at

Отслеживание смен курьеров. Один заказ может иметь несколько курьеров.

delivered_at у снятого курьера = NULL. Записей: 340 649

5. DDL-скрипты

Файл: scripts/init_db.sql

```
CREATE SCHEMA IF NOT EXISTS delivery;

CREATE TABLE delivery.users (
  user_id BIGINT PRIMARY KEY,
  phone   VARCHAR(50) NOT NULL
);
CREATE TABLE delivery.stores (
  store_id   BIGINT PRIMARY KEY,
  store_address VARCHAR(500) NOT NULL
);
CREATE TABLE delivery.drivers (
  driver_id BIGINT PRIMARY KEY,
  phone     VARCHAR(50) NOT NULL
);
CREATE TABLE delivery.items (
  item_id BIGINT PRIMARY KEY,
  title   VARCHAR(500) NOT NULL,
  category VARCHAR(200) NOT NULL
);
CREATE TABLE delivery.orders (
  order_id      BIGINT PRIMARY KEY,
  user_id       BIGINT NOT NULL REFERENCES delivery.users(user_id),
  store_id      BIGINT NOT NULL REFERENCES delivery.stores(store_id),
  address_text  VARCHAR(500) NOT NULL,
  created_at    TIMESTAMP NOT NULL,
  paid_at       TIMESTAMP,
  delivery_started_at TIMESTAMP,
  delivered_at  TIMESTAMP,
  canceled_at   TIMESTAMP,
  payment_type  VARCHAR(50) NOT NULL,
  order_discount INTEGER NOT NULL DEFAULT 0,
  order_cancellation_reason VARCHAR(200),
  delivery_cost INTEGER NOT NULL DEFAULT 0
);
CREATE TABLE delivery.order_items (
  order_id      BIGINT NOT NULL REFERENCES delivery.orders(order_id),
  item_id       BIGINT NOT NULL REFERENCES delivery.items(item_id),
  item_quantity INTEGER NOT NULL,
  item_price    INTEGER NOT NULL,
  item_canceled_quantity INTEGER NOT NULL DEFAULT 0,
  item_replaced_id BIGINT REFERENCES delivery.items(item_id),
  item_discount  INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (order_id, item_id)
);
CREATE TABLE delivery.order_drivers (
  id      SERIAL PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES delivery.orders(order_id),
  driver_id BIGINT NOT NULL REFERENCES delivery.drivers(driver_id),
  delivered_at TIMESTAMP,
  UNIQUE (order_id, driver_id)
);
```

6. ETL-процесс (Airflow)

DAG 1: etl_load_normalized

Читает 34 Parquet-файла по одному (экономия памяти) и загружает в нормализованные таблицы PostgreSQL.

Задачи:

- truncate_tables — очистка всех таблиц (идемпотентность)
- load_dictionaries — загрузка справочников (users, stores, drivers, items)
- load_orders — загрузка 340 000 заказов с дедупликацией
- load_order_items — загрузка 4 212 103 позиций заказов
- load_order_drivers — загрузка 340 649 назначений курьеров

Граф: truncate -> dictionaries -> orders -> [order_items, order_drivers]

Идемпотентность: TRUNCATE + INSERT при каждом запуске.

DAG 2: build_data_marts (PySpark)

Читает таблицы через JDBC, рассчитывает метрики PySpark, записывает в витрины (mode=overwrite). Задачи параллельны.

7. Витрины данных

Витрина заказов (delivery.mart_orders)

Разрезы: год, месяц, день, город, магазин. Записей: 1 101

Колонка	Метрика	Описание
turnover	Оборот	Сумма с учетом скидок
revenue	Выручка	С учетом отмен и скидок
profit	Прибыль	Выручка - delivery_cost
total_orders	Создано заказов	COUNT DISTINCT order_id
delivered_orders	Доставлено	delivered_at IS NOT NULL
canceled_orders	Отменено	canceled_at IS NOT NULL
canceled_after_delivery	Отмен после доставки	canceled + delivered
canceled_service_errors	Ошибки сервиса	Ошибка прил. / Проблемы с оплатой
unique_buyers	Покупателей	COUNT DISTINCT user_id
avg_check	Средний чек	revenue / total_orders
orders_per_buyer	Заказов/покупатель	total / unique_buyers
revenue_per_buyer	Выручка/покупатель	revenue / unique_buyers
driver_changes	Смен курьеров	Заказы с > 1 курьером
active_drivers	Активных курьеров	COUNT DISTINCT driver_id

Витрина товаров (delivery.mart_items)

Разрезы: год, месяц, день, город, магазин, категория, товар. Записей: 19 818

Колонка	Метрика	Описание
item_turnover	Оборот товара	qty * price * (1-disc%) * (1-order_disc%)
ordered_quantity	Заказано единиц	SUM item_quantity
canceled_quantity	Отменено единиц	SUM item_canceled_quantity
orders_with_item	Заказов с товаром	COUNT DISTINCT order_id
orders_with_item_cancel	Заказов с отменой	canceled_qty > 0
is_most_popular_day	Популярный/день	bool — макс. за день
is_least_popular_day	Непопулярный/день	bool — мин. за день
is_most_popular_week	Популярный/неделя	bool — макс. за неделю
is_least_popular_week	Непопулярный/неделя	bool — мин. за неделю
is_most_popular_month	Популярный/месяц	bool — макс. за месяц
is_least_popular_month	Непопулярный/месяц	bool — мин. за месяц